

Python

Знакомство со строками



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

Повторение

- Цикл
- Оператор break
- Оператор continue
- Бесконечный цикл
- Конструкция while/else
- Модуль random

План занятия

- Неизменяемость строк
- Кодировка
- Функции `ord`, `chr`
- Сравнение строк
- Функция `len`
- Индексация строк
- Срезы строк
- Оператор принадлежности `in`



ОСНОВНОЙ БЛОК





Неизменяемость строк



Строки

Это упорядоченные последовательности символов, заключённых в одинарные или двойные кавычки, а также в тройные кавычки (как одинарные, так и двойные).



Задание на повторение

Какой результат выведет следующий код?

```
str1 = "Привет"
str2 = "мир"
result = str1 + str2
print(result)
```

- a. Привет мир
- b. Приветмир
- c. Привет, мир
- d. Ошибка



Задание на повторение

Какой результат выведет следующий код?

```
str1 = "Привет"
str2 = "мир"
result = str1 + str2
print(result)
```

- a. Привет мир
- b. Приветмир**
- c. Привет, мир
- d. Ошибка

Пример неизменяемости строки



```
s = "Hello"
print(s + " world!")
print(s)
```

"Изменение" содержимого строки



```
s = "Hello"  
  
s = s + " world!"  
  
print(s)
```



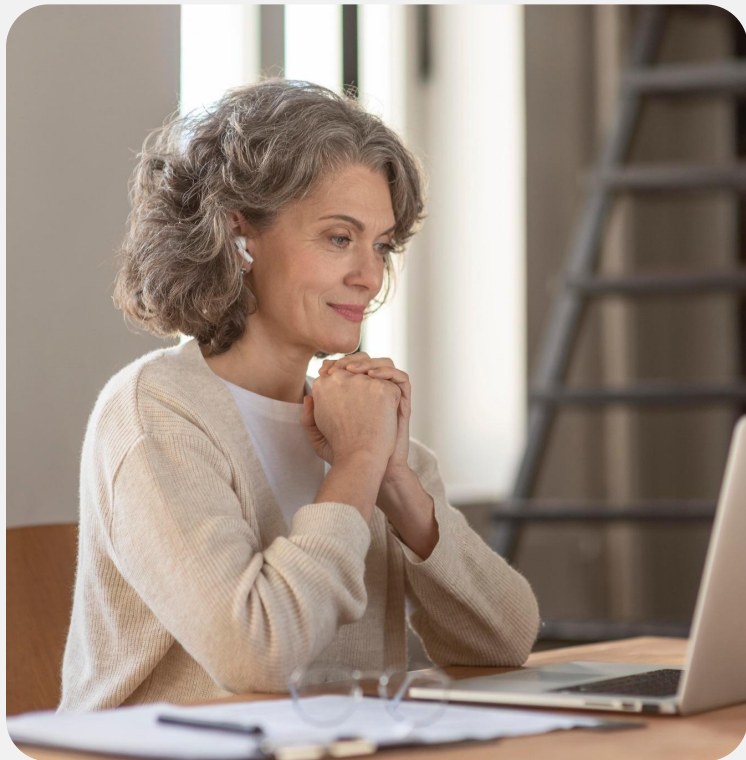
ВОПРОСЫ





ЗАДАНИЕ





Выберите правильный вариант ответа

Какой результат выведет следующий код?

```
s = "Python"
```

```
s += " is fun"
```

```
print(s)
```

- a. Python
- b. Python is fun
- c. Ошибка
- d. Python fun



Выберите правильный вариант ответа

Какой результат выведет следующий код?

```
s = "Python"
```

```
s += " is fun"
```

```
print(s)
```

- a. Python
- b. Python is fun**
- c. Ошибка
- d. Python fun



ВОПРОСЫ





Кодировка





Кодировка

Это способ преобразования символов в числа, которые компьютер может хранить и обрабатывать.



Кодировка ASCII

Это одна из первых и простейших кодировок, разработанная для представления текстовых символов в компьютерах.

Характеристики кодировки ASCII

7 бит

Символы

Простота и совместимость

ASCII использует 7 бит для кодирования символов.

0-31: Управляющие символы (например, символ новой строки, возврат каретки, сигнал окончания текста).

ASCII был создан для работы с английским алфавитом, цифрами и основными символами.

Это позволяет закодировать 128 символов ($2^7 = 128$).

Символы представлены числами от 0 до 127.

32-126: Печатные символы, включая буквы, цифры, пунктуацию и другие символы.

Он широко используется в компьютерах, сетевых системах и других устройствах, обеспечивая базовый уровень кодирования для текстовых данных.

Таблица символов ASCII

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Расширения ASCII

символ	10-Б код	2-Б код	символ	10-Б код	2-Б код	символ	10-Б код	2-Б код	символ	10-Б код	2-Б код
Ъ	128	10000000		160	10100000	А	192	11000000	а	224	11100000
Г	129	10000001	Ѹ	161	10100001	Б	193	11000001	б	225	11100001
,	130	10000010	ѹ	162	10100010	В	194	11000010	в	226	11100010
г	131	10000011	Ј	163	10100011	Г	195	11000011	г	227	11100011
„	132	10000100	Ѱ	164	10100100	Д	196	11000100	д	228	11100100
...	133	10000101	Ѓ	165	10100101	Е	197	11000101	е	229	11100101
†	134	10000110	ђ	166	10100110	Ж	198	11000110	ж	230	11100110
‡	135	10000111	ѕ	167	10100111	З	199	11000111	з	231	11100111
€	136	10001000	Ѕ	168	10101000	И	200	11001000	и	232	11101000
‰	137	10001001	Ѽ	169	10101001	Й	201	11001001	й	233	11101001
Љ	138	10001010	Є	170	10101010	К	202	11001010	к	234	11101010
‹	139	10001011	«	171	10101011	Л	203	11001011	л	235	11101011
Њ	140	10001100	¬	172	10101100	М	204	11001100	м	236	11101100
Ќ	141	10001101	•	173	10101101	Н	205	11001101	н	237	11101101
Њ	142	10001110	®	174	10101110	О	206	11001110	о	238	11101110
Џ	143	10001111	Ї	175	10101111	П	207	11001111	п	239	11101111
ђ	144	10010000	°	176	10110000	Р	208	11010000	р	240	11110000
‘	145	10010001	±	177	10110001	С	209	11010001	с	241	11110001
’	146	10010010	І	178	10110010	Т	210	11010010	т	242	11110010
“	147	10010011	і	179	10110011	У	211	11010011	у	243	11110011
”	148	10010100	г	180	10110100	Ф	212	11010100	ф	244	11110100
•	149	10010101	μ	181	10110101	Х	213	11010101	х	245	11110101
—	150	10010110	¶	182	10110110	Ц	214	11010110	ц	246	11110110
—	151	10010111	·	183	10110111	Ч	215	11010111	ч	247	11110111
□	152	10011000	ё	184	10111000	Ш	216	11011000	ш	248	11111000
™	153	10011001	№	185	10111001	Щ	217	11011001	щ	249	11111001
љ	154	10011010	є	186	10111010	Ъ	218	11011010	ъ	250	11111010
›	155	10011011	»	187	10111011	Ы	219	11011011	ы	251	11111011
њ	156	10011100	ј	188	10111100	Ь	220	11011100	ь	252	11111100
ќ	157	10011101	ѕ	189	10111101	Э	221	11011101	э	253	11111101
ћ	158	10011110	ѕ	190	10111110	Ю	222	11011110	ю	254	11111110
џ	159	10011111	ї	191	10111111	Я	223	11011111	я	255	11111111



Unicode

Это глобальный стандарт, который присваивает каждому символу уникальный числовой код, называемый кодовой точкой.

Категории символов в Unicode



Алфавиты различных языков



Математические и технические символы



Символы для форматирования текста



Эмодзи



Диакритические знаки и другие спецсимволы



Unicode

Таблицу символов Unicode можно посмотреть на сайте

<https://symbl.cc/ru/unicode-table/>



UTF-8

Это одна из самых популярных и широко используемых кодировок для представления символов в Unicode.

Как работает кодировка UTF-8



1 байт: Символы ASCII (от 0 до 127), например, латинские буквы, цифры и некоторые знаки пунктуации.



2 байта: Символы с кодами от 128 до 2047, например, символы латиницы с диакритическими знаками (акценты), кириллические буквы и греческий алфавит.



3 байта: Символы с кодами от 2048 до 65535, включающие китайские иероглифы, японские кандзи и хангиль.



4 байта: Символы с кодами выше 65536, такие как редкие символы и эмодзи.

Преимущества UTF-8:

Эффективное использование
памяти

Для текстов на латинице (английский, испанский и т.д.) UTF-8 экономит память, используя только 1 байт на символ.

Для текстов на других языках, таких как русский или китайский, UTF-8 остаётся эффективной, занимая меньше памяти по сравнению с UTF-16 или UTF-32.

Гибкость

UTF-8 может кодировать символы любой сложности: от простых букв латинского алфавита до редких символов и эмодзи.

Международный стандарт

UTF-8 стал основным стандартом кодирования текстов в Интернете. Большинство веб-страниц и веб-приложений используют UTF-8.



ВОПРОСЫ





ЗАДАНИЕ





Выберите правильный вариант ответа

В чем отличие между кодировками ASCII и UTF-8?

- a. UTF-8 поддерживает больше символов, чем ASCII.
- b. ASCII поддерживает больше символов, чем UTF-8.
- c. В ASCII и UTF-8 одинаковое количество символов.
- d. UTF-8 использует 7 бит для кодирования символов.



Выберите правильный вариант ответа

В чем отличие между кодировками ASCII и UTF-8?

- a. UTF-8 поддерживает больше символов, чем ASCII.
- b. ASCII поддерживает больше символов, чем UTF-8.
- c. В ASCII и UTF-8 одинаковое количество символов.
- d. UTF-8 использует 7 бит для кодирования символов.



Выберите правильный вариант ответа

Какой из следующих диапазонов символов поддерживается кодировкой ASCII?

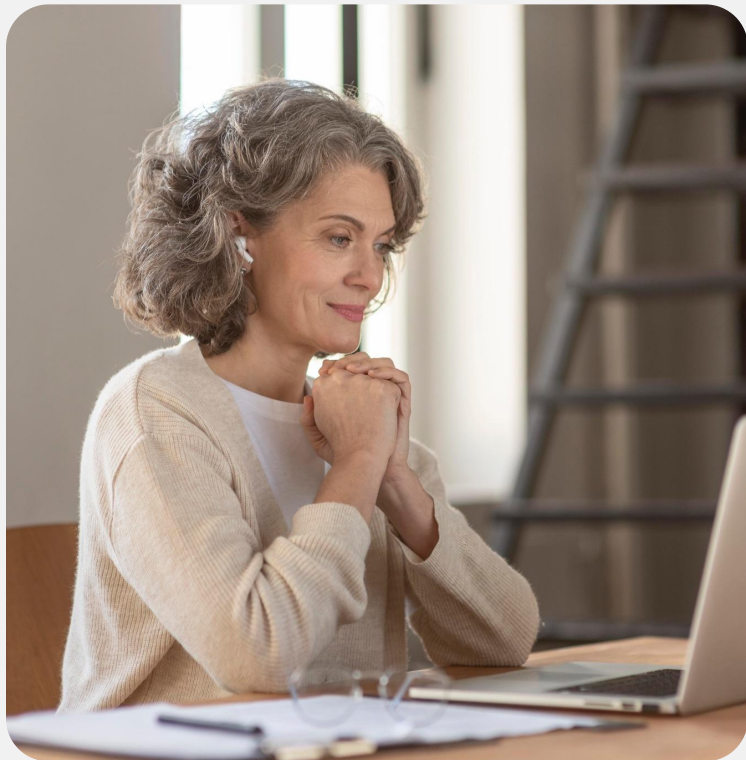
- a. Символы от 0 до 127
- b. Символы от 0 до 255
- c. Символы от 0 до 1024
- d. Символы от 0 до 65535



Выберите правильный вариант ответа

Какой из следующих диапазонов символов поддерживается кодировкой ASCII?

- a. Символы от 0 до 127
- b. Символы от 0 до 255
- c. Символы от 0 до 1024
- d. Символы от 0 до 65535



Выберите правильный вариант ответа

Какой из вариантов неверен относительно Unicode?

- a. Unicode поддерживает символы всех языков мира.
- b. В Unicode содержится больше 143 000 символов.
- c. Unicode использует 1 бит для кодирования всех символов.
- d. Unicode охватывает символы эмодзи и технические знаки.



Выберите правильный вариант ответа

Какой из вариантов неверен относительно Unicode?

- a. Unicode поддерживает символы всех языков мира.
- b. В Unicode содержится больше 143 000 символов.
- c. Unicode использует 1 бит для кодирования всех символов.**
- d. Unicode охватывает символы эмодзи и технические знаки.



ВОПРОСЫ





Функции `ord`, `chr`



Функции `ord`, `chr`

Это встроенные функции Python, которые работают с символами и их кодировкой.



Функция `ord()`

Принимает символ и возвращает его код Unicode. Это полезно, когда нужно узнать числовое значение символа.

Пример



```
print(ord('A'))
```

```
print(ord('a'))
```

```
print(ord(' '))
```



Функция chr()

Принимает число, представляющее код Unicode, и возвращает соответствующий символ.

Пример



```
print(chr(65))
```

```
print(chr(97))
```

```
print(chr(32))
```



ЗАДАНИЕ





Выберите правильный вариант ответа

Что делает функция `ord()` в Python?

- a. Преобразует строку в байты
- b. Возвращает числовой код символа Unicode
- c. Преобразует число в строку
- d. Возвращает кодировку символа



Выберите правильный вариант ответа

Что делает функция `ord()` в Python?

- a. Преобразует строку в байты
- b. Возвращает числовой код символа Unicode**
- c. Преобразует число в строку
- d. Возвращает кодировку символа



Выберите правильный вариант ответа

Какой метод используется для преобразования строки в байты с указанием кодировки?

- a. `decode()`
- b. `ord()`
- c. `encode()`
- d. `chr()`



Выберите правильный вариант ответа

Какой метод используется для преобразования строки в байты с указанием кодировки?

- a. `decode()`
- b. `ord()`
- c. `encode()`**
- d. `chr()`



ВОПРОСЫ





Сравнение строк



Сравнение строк

В Python строки можно сравнивать с помощью операторов сравнения, таких как `==`, `!=`, `<`, `>`, `<=` и `>=`.

Оператор ==



Действие

- Проверяет равенство двух строк.

Код

```
print("apple" == "apple")  
  
print("apple" == "banana")
```

Оператор !=



Действие

- Проверяет неравенство двух строк.

Код

```
print("apple" != "ap ple")
```

Операторы <, >, <=, >=



Действие

- Сравнивают строки лексикографически.
- Строки сравниваются символ за символом на основе их Unicode кодов.

Код

```
print("apple" < "banana")    # True('a' < 'b')
print("Apple" < "banana")    # True ('A' < 'b')
print("apple" > "Apple")      # True ('a' > 'A')
```

Как происходит сравнение



Действие

- **Лексикографический порядок:** Python сравнивает строки символ за символом.
- Как только найдено различие между двумя символами, сравнение завершается.

Код

```
print("abc" < "aca") # True, так как 'b' < 'c'
```

Как происходит сравнение



Действие

- **Учёт регистра:** В Unicode строчные буквы (например, 'a') имеют больший код, чем заглавные (например, 'A').
- Поэтому заглавные буквы считаются "меньшими" по значению.

Код

```
print("apple" > "Apple") # True, так как 'a' > 'A'
```

Как происходит сравнение



Действие

- **Сравнение строк разной длины:** Когда строки разной длины сравниваются, Python продолжает сравнение, пока не закончатся символы в одной из строк.
- Строка, у которой символы закончились первой, считается "меньшей".

Код

```
print("apple" < "app") # False, так как "apple"
не короче
```



ВОПРОСЫ





Функция len



Функция len

Используется для получения длины объектов, таких как строки и другие последовательности. Она возвращает количество элементов в объекте.

Использование функции len() для строк



```
text = "Python"  
length = len(text)  
print(length)
```



ВОПРОСЫ





ЗАДАНИЕ





Выберите правильный вариант ответа

Что произойдет, если сравнить строки разной длины?

```
print("cat" > "catalog")
```

- a. True
- b. False
- c. Ошибка
- d. Невозможно сравнить строки



Выберите правильный вариант ответа

Что произойдет, если сравнить строки разной длины?

```
print("cat" > "catalog")
```

- a. True
- b. False**
- c. Ошибка
- d. Невозможно сравнить строки



Выберите правильный вариант ответа

Какой результат будет выведен при сравнении строк с учётом регистра?

```
print("Zoo" < "zoo")
```

- a. True
- b. False
- c. Ошибка
- d. Невозможно сравнить строки



Выберите правильный вариант ответа

Какой результат будет выведен при сравнении строк с учётом регистра?

```
print("Zoo" < "zoo")
```

- a. **True**
- b. False
- c. Ошибка
- d. Невозможно сравнить строки



Выберите правильный вариант ответа

Какой результат будет выведен при сравнении строк с пробелом?

```
print("apple" != "ap ple")
```

- a. True
- b. False
- c. Ошибка
- d. Невозможно сравнить строки



Выберите правильный вариант ответа

Какой результат будет выведен при сравнении строк с пробелом?

```
print("apple" != "ap ple")
```

- a. **True**
- b. False
- c. Ошибка
- d. Невозможно сравнить строки



Выберите правильный вариант ответа

Какая функция возвращает длину строки?

- a. `length()`
- b. `len()`
- c. `size()`
- d. `count()`



Выберите правильный вариант ответа

Какая функция возвращает длину строки?

- a. length()
- b. len()**
- c. size()
- d. count()



ВОПРОСЫ





Индексация строк



Индексация

Это механизм обращения к отдельным элементам в последовательных структурах данных, таких как строки и другие.

Пример индексации



Для строк:

```
text = "Python"
```

```
print(text[0])    # Вывод: 'P' (первый символ)
```

```
print(text[2])    # Вывод: 't' (третий символ)
```

Отрицательная индексация



```
text = "Python"

print(text[-1]) # Вывод: 'n' (последний символ)

print(text[-3]) # Вывод: 'h' (третий символ с конца)
```

Ошибка при обращении по несуществующему индексу



```
text = "hello"

print(text[10]) # Ошибка, так как в строке "hello" всего 5 символов

print(text[-6]) # Ошибка, так как в строке "hello" всего 5 символов
```

Пример ошибки

```
text = "hello"
print(text[10])
```

IndexError

Traceback (most recent call last)

Cell In[1], line 2

```
1 text = "hello"
----> 2 print(text[10])
```

IndexError: string index out of range

Как избежать ошибки



Действие

- **Проверять длину строки:** Прежде чем обращаться к символу по индексу, можно убедиться, что этот индекс находится в пределах допустимой длины строки.
- Для этого используется функция `len()`, которая возвращает длину строки.

Код

```
text = "hello"
index = 5

if index < len(text):
    print(text[index])

else:
    print("Индекс вне диапазона!")
```



ВОПРОСЫ





Срезы строк





Срезы

Это способ получения подстрок из существующей строки, используя индексы.

Полезно знать



Срезы

- **start** — индекс, с которого начинается срез (включительно).
- **end** — индекс, на котором срез заканчивается (не включительно).

Синтаксис срезов

строка[start:end]

Пример среза строки



```
text = "Python programming"  
print(text[0:6])
```

Если не указывать начальный индекс, срез начнётся с начала строки



```
text = "Python programming"  
print(text[:6])
```

Если не указывать конечный индекс, срез продлится до конца строки



```
text = "Python programming"  
print(text[7:])
```

Если не указывать оба индекса, срез создаст копию всей строки



```
text = "Python programming"  
print(text[:])
```



Отрицательные индексы

Можно использовать отрицательные индексы для срезов с конца строки. Например, индекс -1 соответствует последнему символу строки.

Отрицательные индексы



```
text = "Python programming"  
print(text[-11:])
```



Порядок индексов

При указании индекса нужно учитывать что end должен быть больше чем start.

Пример



```
text = "Python programming"  
  
print(text[-11:-4]) # -11 < -4  
  
print(text[7:14]) # 7 < 14
```

Пример



```
text = "Python programming"
print(text[-4:-11]) # Пустая строка
print(text[14:7]) # Пустая строка
```



Срезы с шагом

Срезы в Python можно использовать не только индексами, но и с шагом (step), который указывает, через сколько символов выбирать элементы.

Полезно знать



Срезы

- **start** — начальный индекс (включительно).
- **end** — конечный индекс (не включительно).
- **step** — шаг, который определяет, через сколько символов брать элементы.

Синтаксис с шагом

строка[start:end:step]

Пример с положительным шагом



```
text = "Python programming"  
print(text[0:12:2]) # Вывод: 'Pto rg'
```

Пример с отрицательным шагом



```
text = "Python programming"  
print(text[12:6:-1]) # Вывод: 'argorp'
```

Порядок индексов при отрицательном шаге



```
text = "Python programming"
print(text[-4:-11:-1]) # -11 < -4
print(text[14:7:-1]) # 7 < 14
```

Пример



```
text = "Python programming"

print(text[-11:-4:-1]) # Пустая строка

print(text[7:14:-1]) # Пустая строка
```

Срез всей строки в обратном порядке



```
text = "Python"  
print(text[::-1]) # Вывод: 'nohtyP'
```



ВОПРОСЫ





Оператор _____
принадлежности in



Оператор in

Используется для проверки, содержит ли строка определённый элемент или подстроку.

Пример с использованием оператора in для строк



```
text = "Python programming"
print("Python" in text)
print("Hello" in text)
```



Комбинация not и in

Используется для проверки, что элемент не содержится в строке или другой последовательности. not просто инвертирует результат выражения с оператором in.

Пример



```
text = "Python programming"
```

```
print("Java" not in text)
```



ВОПРОСЫ





ЗАДАНИЕ





Выберите правильный вариант ответа

Какой результат будет выведен?

```
text = "Hello, Python!"
```

```
print(text[7])
```

- a. H
- b. o
- c. P
- d. Ошибка



Выберите правильный вариант ответа

Какой результат будет выведен?

```
text = "Hello, Python!"
```

```
print(text[7])
```

- a. H
- b. o
- c. P**
- d. Ошибка



Выберите правильный вариант ответа

Какой результат будет выведен?

```
text = "Hello, Python!"
```

```
print(text[-3])
```

- a. t
- b. o
- c. h
- d. o



Выберите правильный вариант ответа

Какой результат будет выведен?

```
text = "Hello, Python!"
```

```
print(text[-3])
```

- a. t
- b. o
- c. h
- d. o**



Выберите правильный вариант ответа

Какой результат будет выведен?

```
text = "Hello, Python!"
```

```
print(text[1:6])
```

- a. Hello
- b. ello,
- c. ello
- d. Ошибка



Выберите правильный вариант ответа

Какой результат будет выведен?

```
text = "Hello, Python!"
```

```
print(text[1:6])
```

- a. Hello
- b. ello,**
- c. ello
- d. Ошибка



Выберите правильный вариант ответа

Какой результат будет выведен?

```
text = "Hello"
```

```
print(text[10])
```

- Вывод символа с индексом 10
- Вывод первого символа
- Ошибка
- Вывод последнего символа



Выберите правильный вариант ответа

Какой результат будет выведен?

```
text = "Hello"
```

```
print(text[10])
```

- a. Вывод символа с индексом 10
- b. Вывод первого символа
- c. Ошибка**
- d. Вывод последнего символа



Выберите правильный вариант ответа

Какой результат будет выведен?

```
text = "Python"
```

```
print(text[::-1])
```

- a. Python
- b. nohtyP
- c. nohty
- d. Ошибка



Выберите правильный вариант ответа

Какой результат будет выведен?

```
text = "Python"
```

```
print(text[::-1])
```

- a. Python
- b. nohtyP**
- c. nohty
- d. Ошибка



Выберите правильный вариант ответа

Какой результат будет выведен?

```
text = "Python programming"
```

```
print("python" in text)
```

- a. True
- b. False
- c. Ошибка
- d. None



Выберите правильный вариант ответа

Какой результат будет выведен?

```
text = "Python programming"
```

```
print("python" in text)
```

- a. True
- b. False**
- c. Ошибка
- d. None



ПРАКТИЧЕСКАЯ РАБОТА



1. Проверка палиндрома



Напишите программу, которая проверяет является ли строка палиндромом. Палиндром - это строка, которая читается одинаково слева направо и справа налево).

Пример вывода:

Введите строку: radar

Строка является палиндромом

2. Шифр Цезаря



Напишите программу для дешифровки текста, зашифрованного с помощью шифра Цезаря. Программа принимает зашифрованный текст и сдвиг (использованный для зашифровки) от пользователя и выводит расшифрованный текст.

Шифр Цезаря — это один из самых простых и известных методов шифрования. В этом методе каждый символ в исходном тексте заменяется на символ, находящийся на фиксированное число позиций дальше в алфавите. Сдвиг может быть как влево, так и вправо, и для расшифровки необходимо просто сдвинуть символы обратно на тот же шаг.

Пример вывода:

Введите зашифрованный текст: `khoor`

Введите сдвиг: `3`

Расшифрованный текст: `hello`



ВОПРОСЫ



Домашнее задание

1. Проверка кодировки

Напишите программу, которая принимает от пользователя один символ и выводит его код в таблице Unicode и его принадлежность к диапазону ASCII (True/False).

Пример вывода:

Введите символ: A

Код Unicode: 65

ASCII символ: True

Домашнее задание

2. Подстрока с заполнением

Напишите программу, которая принимает строку и индексы начала и конца. Программа должна вывести подстроку на указанных позициях. Также если конечный индекс выходит за пределы строки, программа заполняет недостающие символы символами _.

Пример вывода:

Введите строку: Программирование

Введите начальный индекс: 3

Введите конечный индекс: 20

Подстрока: граммирование_____

Домашнее задание

3. Подсчёт символа

Напишите программу, которая принимает строку и символ, а затем подсчитывает, сколько раз символ встречается в строке.

Пример вывода:

Введите строку: Hello, world!

Введите символ: o

Символ o встречается 2 раз(а).

Заключение

